ГУАП

КАФЕДРА № 41

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

Ассистент		А.С. Раскопина
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

Создание запросов на сервере MySQL

по курсу: Базы данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № 4417		М. П. Курбатов
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

Лабораторная работа № 6. Программная реализация хранимых процедур

Цель работы

Реализовать одну хранимую процедуру для базы «ГИБДД» с входным параметром, ветвлением, циклом, локальными и пользовательскими сессионными переменными. Проверить допустимые и ошибочные параметры.

1 Вариант и база данных

Вариант 13 — «ГИБДД». Инспекция дорожного движения Хоббитонии регистрирует автомобили граждан и учитывает ДТП. У автомобиля хранятся марка, мощность, максимальная скорость, год выпуска, стоимость и текущий владелец. Для ДТП фиксируются дата, время, адрес и участвующие автомобили. Один гражданин может владеть несколькими автомобилями и продавать их другим гражданам.

Используется база gibdd из работ № 4–5. Контрольный набор включает 10 граждан, 10 автомобилей, 5 ДТП, 10 участий и 5 продаж. В таблице 2 приведены контрольные значения для проверки процедуры. Результаты выполнения в MySQL Workbench показаны на рисунках 3–9.

На рисунке 1 показана схема базы gibdd из пяти таблиц. Шесть внешних ключей связывают граждан, автомобили, ДТП, участия и продажи. Связи перечислены в таблице 1.

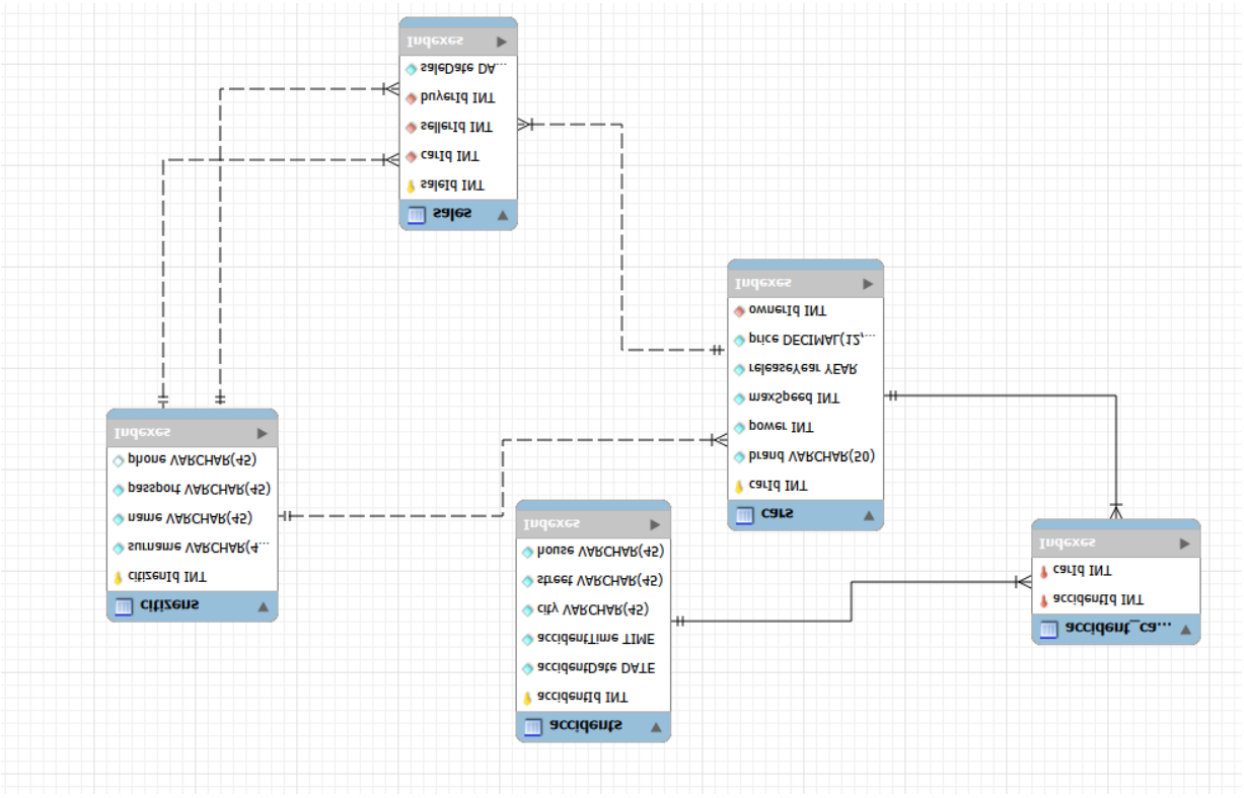


Рисунок 1 — Схема данных базы «ГИБДД» в MySQL Workbench

Таблица 1 — Связи базы данных

Внешний ключ	**Родительский ключ**
cars.ownerId	citizens.citizenId
accident_cars.accidentId	accidents.accidentId
accident_cars.carId	cars.carId
sales.carId	cars.carId
sales.sellerId	citizens.citizenId
sales.buyerId	citizens.citizenId

В accident_cars составной первичный ключ (accidentId, carId). Он не допускает повторения одного автомобиля в одном ДТП. В этой таблице считаются участия, а в accidents — сами происшествия. Число участий не определяет виновность водителя.

2 Назначение и алгоритм процедуры

Процедура lr6_simple_year(p_year) нужна аналитику ГИБДД, чтобы узнать общее число зарегистрированных ДТП за выбранный год. IN p_year INT — входной год от 1901 до 2155 включительно. Нормальный результат —

одна строка report_year и accident_count. Если год неверен или NULL, возвращается строка message с текстом ERROR: year must be 1901..2155. Это обычный результат SELECT, а не SQL-исключение.

Локальные переменные: v_month — номер месяца, v_count — число ДТП текущего месяца, v_total — накопленная сумма. Пользовательская @lr6_total хранит итог после CALL в том же соединении. Такие переменные в учебной терминологии называют глобальными по отношению к блоку процедуры; технически их область действия — текущая сессия. Настройки @@GLOBAL сервера не меняются.

Алгоритм на рисунке 2: сбросить прежний сессионный итог; проверить год; при ошибке вывести сообщение; иначе перебрать 12 месяцев, посчитать ДТП каждого месяца и сложить результаты; записать итог в @lr6_total и вывести его. Нулевой результат означает допустимый год без ДТП. NULL в @lr6_total после ошибочного вызова не позволяет спутать ошибку с отсутствием происшествий.

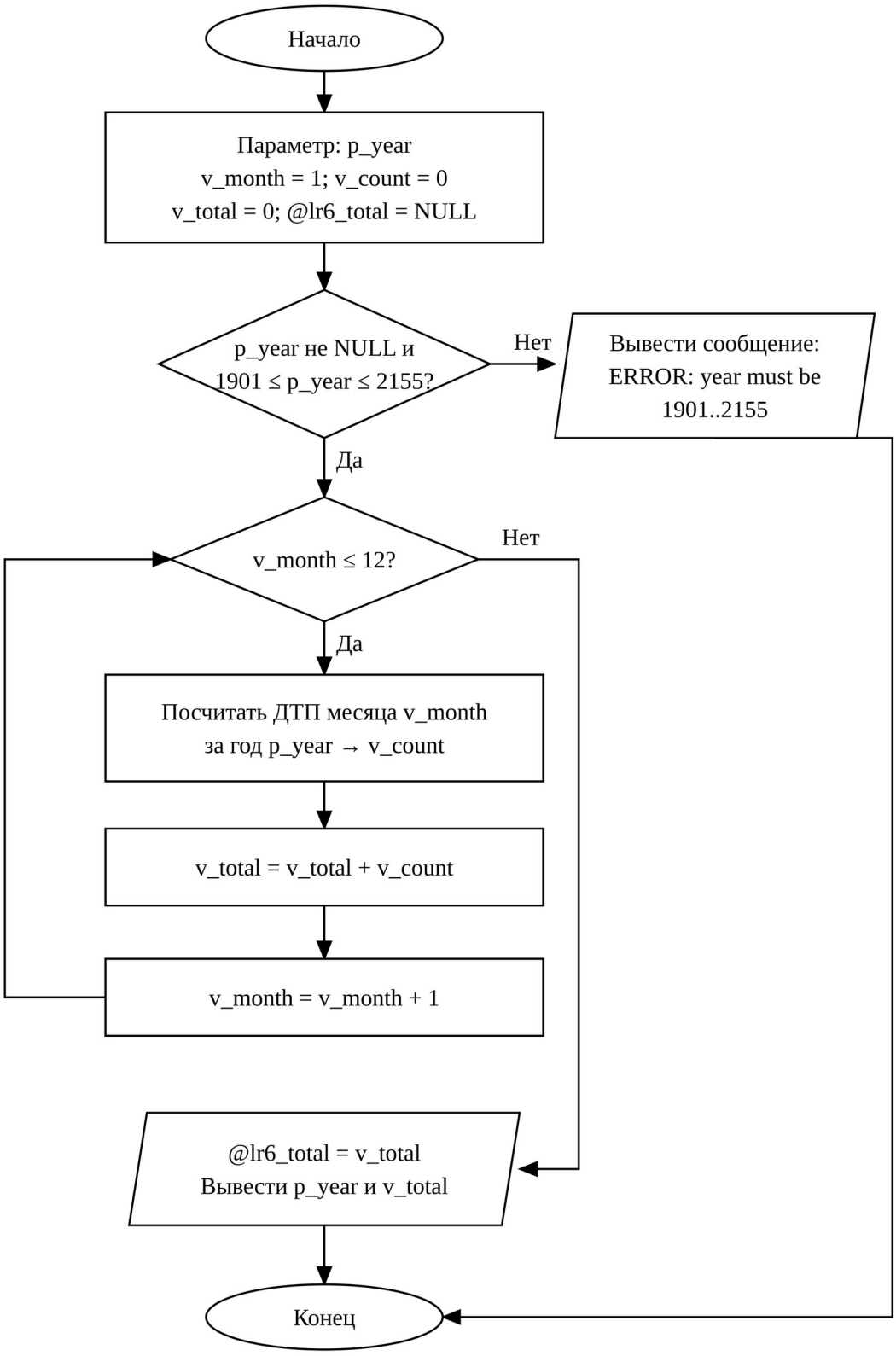


Рисунок 2 — Блок-схема алгоритма процедуры lr6_simple_year

3 Листинг и пояснения

В листинге 1 приведён полный установочный файл. Четыре комментария отмечают основные действия. Редактируемая блок-схема

рисунка 2 приложена в файле «Алгоритм_ЛР6.drawio». Перебор месяцев выбран для выполнения требования методички о цикле. Для небольшого учебного набора YEAR и MONTH делают фильтр понятным; на большой базе способ подсчёта следует оценить отдельно.

Листинг 1 — Создание простой процедуры

```
USE gibdd;
SET NAMES utf8mb4;
DROP PROCEDURE IF EXISTS lr6_simple_year;
DELIMITER $$
CREATE PROCEDURE lr6_simple_year(IN p_year INT)
READS SQL DATA
BEGIN
  DECLARE v_month INT DEFAULT 1;
  DECLARE v_count INT DEFAULT 0;
  DECLARE v_total INT DEFAULT 0;
  -- 1. Сбрасываем прошлый сессионный итог.
  SET @lr6_total = NULL;
  -- 2. Проверяем входной год.
  IF p_year IS NULL
     OR p_year < 1901 OR p_year > 2155 THEN
    SELECT
      'ERROR: year must be 1901..2155'
      AS message;
  ELSE
    -- 3. Считаем ДТП каждого из 12 месяцев.
    WHILE v_month <= 12 DO
      SELECT COUNT(*) INTO v_count
      FROM gibdd.accidents
      WHERE YEAR(accidentDate) = p_year
        AND MONTH(accidentDate) = v_month;
      SET v_total = v_total + v_count;
      SET v_month = v_month + 1;
```

```
    END WHILE;
    -- 4. Сохраняем и выводим итог.
    SET @lr6_total = v_total;
    SELECT p_year AS report_year,
           v_total AS accident_count;
  END IF;
END$$
DELIMITER ;
```

DECLARE объявляет локальные переменные до исполняемых операторов. SELECT COUNT(*) INTO v_count помещает число строк в переменную. SET v_total = v_total + v_count накапливает итог; SET v_month = v_month + 1 обеспечивает завершение цикла. DELIMITER меняет разделитель только для клиента при создании процедуры.

4 Список процедур и тестирование

Запрос листинга 2 выводит список хранимых процедур базы gibdd. На рисунке 3 показан результат выполненного запроса: в базе присутствует процедура lr6_simple_year.

Листинг 2 — Список процедур

```
SELECT ROUTINE_NAME, ROUTINE_TYPE
FROM information_schema.ROUTINES
WHERE ROUTINE_SCHEMA = 'gibdd'
  AND ROUTINE_TYPE = 'PROCEDURE'
ORDER BY ROUTINE_NAME;
```

```
SELECT ROUTINE_NAME, ROUTINE_TYPE
FROM information_schema.ROUTINES
WHERE ROUTINE_SCHEMA = 'gibdd'
  AND ROUTINE_TYPE = 'PROCEDURE'
ORDER BY ROUTINE_NAME;
```

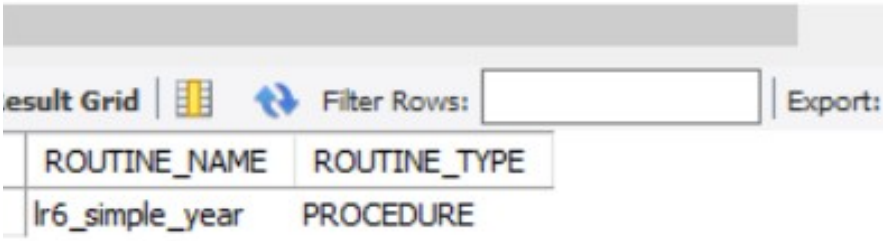


Рисунок 3 — Результат запроса списка хранимых процедур базы gibdd

В таблице 2 приведены контрольные значения для положительных, граничных и отрицательных тестов. В листинге 3 записаны соответствующие вызовы. CALL и следующий SELECT @lr6_total выполняются в одном соединении. На рисунках 4 и 5 показаны сессионные итоги для 2025 и 2024 годов; на рисунке 6 — результат вызова для верхнего допустимого года 2155.

Таблица 2 — Ожидаемые результаты процедуры

p_year	**Результат CALL**	**@lr6_total**
2025	2025; 5 ДТП	5
2024	2024; 0 ДТП	0
1901 / 2155	допустимые границы; по 0 ДТП	0
NULL / 1900 / 2156	сообщение ERROR о неверном годе	NULL

Листинг 3 — Вызовы процедуры

```
USE gibdd;
-- Рисунок 3: список всех процедур базы.
SELECT ROUTINE_NAME, ROUTINE_TYPE
FROM information_schema.ROUTINES
WHERE ROUTINE_SCHEMA = 'gibdd'
  AND ROUTINE_TYPE = 'PROCEDURE'
ORDER BY ROUTINE_NAME;
-- Рисунок 4: год с ДТП; ожидается 5.
CALL lr6_simple_year(2025);
SELECT @lr6_total AS total;
-- Рисунок 5: год без ДТП; ожидается 0.
CALL lr6_simple_year(2024);
SELECT @lr6_total AS total;
-- Рисунок 6: допустимые границы.
CALL lr6_simple_year(1901);
CALL lr6_simple_year(2155);
-- Рисунок 7: неверные годы; итог NULL.
-- Каждый CALL и SELECT запускать отдельно.
CALL lr6_simple_year(NULL);
SELECT @lr6_total AS total;
CALL lr6_simple_year(1900);
SELECT @lr6_total AS total;
CALL lr6_simple_year(2156);
SELECT @lr6_total AS total;
```

```
8     -- Рисунок 4: год с ДТП; ожидается 5.
9     CALL lr6_simple_year(2025);
10    SELECT @lr6_total AS total;
11
12
```

Result Grid | Filter Rows: | Export:

total
5

Рисунок 4 — Значение @lr6_total после вызова процедуры для 2025 года

```
1     USE gibdd;
2     -- Рисунок 5: год без ДТП; ожидается 0.
3     CALL lr6_simple_year(2024);
4     SELECT @lr6_total AS total;
5
```

Result Grid | Filter Rows: | Export:

total
0

Рисунок 5 — Значение @lr6_total после вызова процедуры для 2024 года

```
1     USE gibdd;
2     -- Рисунок 6: допустимые границы.
3     CALL lr6_simple_year(1901);
4     CALL lr6_simple_year(2155);
5
```

Result Grid | Filter Rows: | Export: | Wra

report_year	accident_count
2155	0

Рисунок 6 — Результат вызова процедуры для верхнего допустимого года 2155

```
4 •   CALL lr6_simple_year(NULL);
5 •   SELECT @lr6_total AS total;
6 •   CALL lr6_simple_year(1900);
7 •   SELECT @lr6_total AS total;
8 •   CALL lr6_simple_year(2156);
9 •   SELECT @lr6_total AS total;
10
```

Result Grid | Filter Rows: | Export:

message
ERROR: year must be 1901..2155

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total
NULL

Рисунок 7 — Первый отрицательный тест: сообщение о неверном годе и сессионный итог NULL

Result Grid | Filter Rows: | Export: | Wrap Cell Conter

message
ERROR: year must be 1901..2155

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

total
NULL

Рисунок 8 — Повторный отрицательный тест: сообщение о неверном годе и сессионный итог NULL

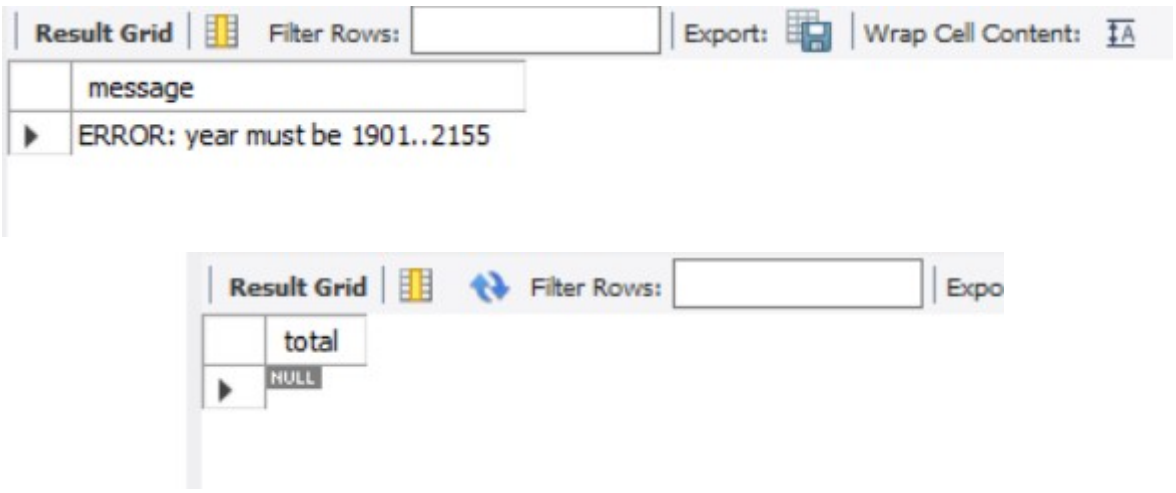


Рисунок 9 — Третий отрицательный тест: сообщение о неверном годе и сессионный итог NULL

На рисунках 7–9 показаны результаты отрицательного тестирования: процедура возвращает сообщение о недопустимом годе, а сессионный итог принимает значение NULL. Таким образом, продемонстрированы успешная ветвь IF и ветвь обработки ошибочного параметра. Значение NULL отличается от нулевого количества ДТП за допустимый год. Процедура использует только чтение таблицы accidents и не содержит команд изменения исходных записей.

5 Вывод

Разработанная процедура объединяет простой запрос к данным с программным алгоритмом. Аналитик передаёт год и получает одну строку с итогом. Ему не требуется каждый раз писать фильтр по дате, а проверка параметра помогает отличить ошибку ввода от года без происшествий. Для ГИБДД это полезно при подготовке годовой сводки.

Цикл последовательно обходит месяцы и складывает их показатели. На контрольном наборе в августе 2025 года зарегистрировано одно ДТП, в сентябре и октябре — по два, поэтому сумма равна пяти. Поскольку подсчёт выполняется по accidents, число автомобилей-участников не увеличивает итог. Это соответствует смыслу показателя «число ДТП».

Локальные переменные сохраняют промежуточные значения только во время вызова. @lr6_total позволяет прочитать окончательный результат после

CALL в том же соединении. Сброс этой переменной до проверки параметра предотвращает использование старого итога при ошибке. Допустимые, граничные и ошибочные вызовы показывают обе ветви алгоритма. Процедура только читает базу, поэтому учебные данные сохраняются.

Список использованных источников

1. Аграновский А. В., Боженко В. В., Турнецкая Е. Л. Разработка и администрирование базы данных с открытым исходным кодом. ГУАП, 2022. Подраздел 7.5, с. 131–133; приложение В, вариант 13.

2. MySQL 8.0 Reference Manual: CREATE PROCEDURE, CREATE FUNCTION, IF, WHILE, RETURN, Local Variables, User-Defined Variables. URL: https://dev.mysql.com/doc/refman/8.0/en/.

3. Курбатов М. П. Отчёт о лабораторной работе № 5 «Создание запросов на сервере MySQL». Предоставленный исходный отчёт, 2026.

4. ГОСТ 7.32–2017. Отчёт о научно-исследовательской работе. ГОСТ Р 2.105–2019. Общие требования к текстовым документам.